REPO DOCTOR: Complete System Reference

Version: 3.0
Protocol: MERMEDA (Modular Extensible Repository Management, Enhancement, Detection, and Automation)
Built by: CyberAI Oracle Network
Contact: V3@CyberAi.network

Table of Contents

1. Executive Overview
2. System Architecture
3. The 29-Phase MERMEDA Pipeline
4. The 19-Phase Operational Loop
5. GitHub Workflows
6. Core Shell Modules
7. Governance Rules
8. Codebooks
9. Addons & Enhancements
10. UI/UX Constraints
11. Temporal Architecture
12. Fleet Governance
13. Customization Guide
14. Troubleshooting
15. Implementation Checklist
16. Compliance & Certification
17. API Reference
18. Appendices

Executive Overview

What is Repo Doctor?

Repo Doctor is a **self-governing, autonomous repair engine** for software repositories. It operates as a 29-

phase MERMEDA pipeline that moves repositories from "Unknown/Drifted" states to "Green-Locked/Nominal" through systematic detection, repair, and governance.

Core Capabilities

- **Polyglot Detection**: Automatically identifies Node.js, Python, Rust, Solidity, and hybrid stacks
- **Framework Intelligence**: Detects and configures Next.js, Angular, Nuxt, Astro, and 20+ frameworks
- **Autonomous Repair**: Creates remediation branches and PRs without human intervention
- **Security Forensics**: Scans for API key leaks, RCE vulnerabilities, and unsafe patterns
- **Fleet Management**: Orchestrates governance across entire GitHub organizations
- **AI Governance**: Generates behavioral contracts for Copilot and Cursor
- **Zero-Touch Operation**: Fully automated via GitHub Actions

Key Design Principles

1. **Transparency**: All logic stored in visible `repo-brain/` directory
2. **Idempotency**: All phases can be run repeatedly without side effects
3. **Determinism**: Same input always produces same output
4. **Non-Invasiveness**: Safe operations first, structural repairs require review
5. **Auditability**: Every action logged and traceable

System Architecture

Directory Structure

```
repository/
├── .github/
│   ├── workflows/
│   │   ├── brain.yml                # Master admission workflow
│   │   ├── brain-doctor.yml         # Self-healing workflow
│   │   ├── brain-auto-comments.yml  # PR advisory system
│   │   ├── next-ci.yml              # Next.js canonical template
│   │   ├── solidity-ci.yml          # Web3 canonical template
│   │   └── angular-ci.yml           # Angular canonical template
│   └── copilot-instructions.md      # AI governance contract
├── repo-brain/
│   ├── brain.run.sh                 # Master orchestrator
│   ├── brainctl.sh                  # CLI interface
│   ├── brain.detect.sh              # Language fingerprinting
```

```
│   ├── brain.scan-actions.sh          # CI/CD audit
│   ├── brain.frameworks.sh            # Framework detection
│   ├── brain.frameworks.ci.sh         # CI negotiation
│   ├── brain.solidity.detect.sh       # Web3 discovery
│   ├── brain.solidity.ci.sh           # Smart contract CI
│   ├── brain.forge.repair.sh          # Foundry surgeon
│   ├── brain.rust.sh                  # Rust synchronization
│   ├── brain.normalize.sh             # Genome injection
│   ├── brain.vercel.troubleshoot.sh   # Deployment specialist
│   ├── brain.diagnose.sh              # Health scoring
│   ├── brain.fix.safe.sh              # Non-invasive patching
│   ├── brain.verify.sh                # Motor function check
│   ├── brain.ai.guard.sh              # Forensic security
│   ├── brain.greenlock.sh             # Integrity sealing
│   ├── brain.guard.sh                 # Neural contract
│   ├── brain.auto-pr.sh               # Auto-wiring engine
│   ├── brain.fleet.sh                 # Telemetry sync
│   ├── brain.test.sh                  # Graduation suite
│   ├── brain.doctor.sh                # Self-health audit
│   ├── brain.surgeon.sh               # Genomic restoration
│   ├── brain.immunizer.sh             # Integrity lock
│   ├── brain.firewall.sh              # Pre-commit denial
│   ├── brain.vitals.sh                # Metric collector
│   ├── brain.autopsy.sh               # Forensic tracer
│   ├── brain.patch.sh                 # Structural patching
│   ├── brain.repair.deep.sh           # High-risk advisory
│   ├── brain.fleet.doctor.sh          # Fleet drift detection
│   ├── brain.fleet.surgeon.sh         # Fleet restoration
│   ├── brain.oracle.sh                # Oracle verdict
│   ├── copilot.sh                     # Neural graft utility
│   └── fleet-bootstrap.sh             # Org-wide rollout
├── diagnosis.json                     # Health report (generated)
├── brain.fleet.json                   # Fleet telemetry (generated)
└── .repo-brain.lock                   # SHA-256 genome lock (generated)
```

Data Flow Architecture

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    GitHub Event Trigger                                  │
│                (push, PR, schedule, manual)                              │
└──────────────────────────────────────────────────────────────────────────┘
                    │
                    ▼
```

```
┌──────────────────────────────────────────────────────────────┐
│              brain.yml (Master Workflow)                     │
│  • Environment Setup (Node.js, Foundry, Rust, Python)        │
│  • Execute brain.run.sh (19-phase loop)                      │
└──────────────────────────────────────────────────────────────┘
                │
                ▼
┌──────────────────────────────────────────────────────────────┐
│         brain.run.sh (Orchestrator - 19 Phases)              │
│  Phase 01-08: Detection & Discovery                          │
│  Phase 09-13: Normalization & Safe Repair                    │
│  Phase 14-16: Security & Governance                          │
│  Phase 17-19: Autonomy & Validation                          │
└──────────────────────────────────────────────────────────────┘
                │
                ▼
┌──────────────────────────────────────────────────────────────┐
│           diagnosis.json (Health Report)                     │
│  • Score: 1-100                                              │
│  • State: GREEN / YELLOW / RED / AUTO                        │
│  • Findings: Array of issues and repairs                     │
└──────────────────────────────────────────────────────────────┘
                │
      ┌─────────┴─────────┬─────────┐
      │         │         │
      ▼         ▼         ▼
┌─────────────┐ ┌─────────────┐ ┌─────────────┐
│ Auto-PR     │ │ Fleet Sync  │ │ AI Guard    │
│ Creation    │ │ Update      │ │ Advisory    │
└─────────────┘ └─────────────┘ └─────────────┘
```

Execution Modes

1. **Admission Mode**: Full 19-phase pipeline on new repositories
2. **Maintenance Mode**: Continuous monitoring via GitHub Actions
3. **Repair Mode**: Targeted fixes via `brainctl` commands
4. **Fleet Mode**: Org-wide governance and synchronization
5. **Oracle Mode**: Final validation and certification

The 29-Phase MERMEDA Pipeline

The complete MERMEDA protocol extends the 19-phase operational loop with 10 additional deep repair, validation, and fleet governance phases.

Phase Categorization

Detection Layer (Phases 1-8)

- Language and toolchain discovery
- Framework and dependency mapping
- CI/CD infrastructure audit

Normalization Layer (Phases 9-13)

- Canonical template injection
- Safe patching and repair
- Build/test contract negotiation

Security Layer (Phases 14-16)

- Forensic scanning
- Integrity sealing
- AI governance contract generation

Autonomy Layer (Phases 17-22)

- Auto-repair and PR creation
- Structural patching
- Deep repair advisory

Fleet Layer (Phases 23-26)

- Telemetry aggregation
- Drift detection
- Cross-repo synchronization

Oracle Layer (Phases 27-29)

- Final validation
- Certification
- Immutability enforcement

Complete 29-Phase Breakdown

Phase	Module	Category	Risk Level	Description
01	brain.detect.sh	Detection	None	Language fingerprinting: Node, Python, Rust, Solidity
02	brain.scan-actions.sh	Detection	None	CI/CD audit: Maps existing workflows
03	brain.frameworks.sh	Detection	None	Framework detection: Next.js, Angular, Nuxt, etc.
04	brain.frameworks.ci.sh	Detection	None	CI negotiation: Determines build/test commands
05	brain.solidity.detect.sh	Detection	None	Web3 discovery: Foundry/Hardhat detection
06	brain.solidity.ci.sh	Detection	None	Smart contract CI contract preparation
07	brain.forge.repair.sh	Detection	Low	Forge surgeon: Standardizes remappings
08	brain.rust.sh	Detection	None	Rust synchronization: Cargo.toml audit
09	brain.normalize.sh	Normalization	Low	Genome injection: Deploys canonical templates
10	brain.vercel.troubleshoot.sh	Normalization	Low	Deployment specialist: OOM detection
11	brain.diagnose.sh	Normalization	None	Health scoring: Generates diagnosis.json
12	brain.fix.safe.sh	Normalization	Low	Non-invasive patching: .gitignore, permissions
13	brain.verify.sh	Normalization	None	Motor function check: Build/test execution
14	brain.ai.guard.sh	Security	None	Forensic security: Secret/injection detection
15	brain.greenlock.sh	Security	None	Integrity sealing: SHA-256 lock application
16	brain.guard.sh	Security	None	Neural contract: Copilot governance
17	brain.auto-pr.sh	Autonomy	Medium	Auto-wiring: Creates remediation PRs
18	brain.fleet.sh	Autonomy	None	Telemetry sync: Updates fleet.json
19	brain.test.sh	Autonomy	None	Graduation suite: Regression validation

Phase	Module	Category	Risk Level	Description
20	brain.patch.sh	Deep Repair	Medium	Structural patching: Directory restructuring
21	brain.repair.deep.sh	Deep Repair	High	High-risk repair: Requires human review
22	brain.immunizer.sh	Deep Repair	Low	Genome integrity lock and verification
23	brain.fleet.doctor.sh	Fleet	None	Fleet drift detection across org
24	brain.fleet.surgeon.sh	Fleet	Medium	Fleet genome restoration
25	brain.vitals.sh	Fleet	None	Metric collection and aggregation
26	brain.autopsy.sh	Fleet	None	Forensic trace for failed builds
27	brain.oracle.sh	Oracle	None	Final validation and certification
28	brain.doctor.sh	Oracle	None	Self-health audit of brain modules
29	brain.surgeon.sh	Oracle	Low	Genomic restoration of brain itself

Risk Level Definitions

- **None**: Read-only operations, no file modifications
- **Low**: Safe file modifications (configs, gitignore, permissions)
- **Medium**: Structural changes (directories, CI templates, automated PRs)
- **High**: Invasive repairs requiring human review and approval

---

The 19-Phase Operational Loop

This is the standard execution sequence performed by brain.run.sh during repository admission.

Phase 01: Language Fingerprinting

Module: brain.detect.sh
Purpose: Establish the repository's base genome
Risk: None (read-only)

Detection Signatures:

```bash
```

```
# Node.js Detection
- package.json
- node_modules/
- .nvmrc, .node-version
- npm-shrinkwrap.json, package-lock.json, yarn.lock, pnpm-lock.yaml

# Python Detection
- setup.py, pyproject.toml, requirements.txt
- Pipfile, poetry.lock
- __pycache__/, *.pyc
- venv/, .venv/, virtualenv/

# Rust Detection
- Cargo.toml, Cargo.lock
- src/main.rs, src/lib.rs
- target/

# Solidity Detection
- contracts/*.sol
- foundry.toml, hardhat.config.js, hardhat.config.ts
- remappings.txt
- lib/, node_modules/@openzeppelin/
```

Output Format:

```json
{
  "languages": ["node", "solidity"],
  "primary": "node",
  "confidence": 0.95,
  "signatures": {
    "node": ["package.json", "node_modules"],
    "solidity": ["contracts/", "foundry.toml"]
  }
}
```

Customization Points:

- Add custom language signatures in `LANGUAGE_PATTERNS` array
- Adjust confidence thresholds in `CONFIDENCE_WEIGHTS`
- Override primary language detection logic

Phase 02: CI/CD Audit

Module: brain.scan-actions.sh
Purpose: Map existing automation to prevent conflicts
Risk: None (read-only)

Scanned Elements:

```yaml
# Workflow Discovery
.github/workflows/*.yml
.github/workflows/*.yaml

# Analyzed Properties
- name: Workflow display name
- on: Trigger events (push, pull_request, schedule)
- jobs: Job names and steps
- uses: Action dependencies
- run: Shell commands
```

Conflict Detection:

```bash
# Overlapping Build Jobs
- Multiple "build" jobs
- Duplicate "test" jobs
- Conflicting "deploy" jobs

# Dependency Conflicts
- Different Node.js versions across workflows
- Incompatible cache strategies
- Redundant setup steps
```

Output Format:

```json
```

```
{
  "workflows": [
    {
      "file": ".github/workflows/ci.yml",
      "name": "CI Pipeline",
      "triggers": ["push", "pull_request"],
      "jobs": ["build", "test", "lint"],
      "conflicts": []
    }
  ],
  "recommendations": [
    "Consolidate 'build' and 'test' into single workflow",
    "Standardize Node.js version to 20.x"
  ]
}
```

Phase 03: Framework Detection

Module: brain.frameworks.sh
Purpose: Identify higher-order frameworks
Risk: None (read-only)

Supported Frameworks:

```javascript
```

```
// Frontend Frameworks
{
  "next": {
    "signatures": ["next.config.js", "pages/", "app/"],
    "deps": ["next"],
    "version_key": "next"
  },
  "nuxt": {
    "signatures": ["nuxt.config.js", "nuxt.config.ts"],
    "deps": ["nuxt"],
    "version_key": "nuxt"
  },
  "angular": {
    "signatures": ["angular.json"],
    "deps": ["@angular/core"],
    "version_key": "@angular/core"
  },
  "astro": {
    "signatures": ["astro.config.mjs"],
    "deps": ["astro"],
    "version_key": "astro"
  },
  "svelte": {
    "signatures": ["svelte.config.js"],
    "deps": ["svelte"],
    "version_key": "svelte"
  },
  "vue": {
    "signatures": ["vue.config.js", "App.vue"],
    "deps": ["vue"],
    "version_key": "vue"
  }
}

// Backend Frameworks
{
  "express": {
    "signatures": ["server.js", "app.js"],
    "deps": ["express"]
  },
  "nest": {
    "signatures": ["nest-cli.json"],
    "deps": ["@nestjs/core"]
```

```
  },
  "fastify": {
    "deps": ["fastify"]
  }
}

// Web3 Libraries
{
  "ethers": {
    "deps": ["ethers"]
  },
  "web3": {
    "deps": ["web3"]
  },
  "viem": {
    "deps": ["viem"]
  },
  "wagmi": {
    "deps": ["wagmi"]
  }
}
```

Detection Algorithm:

```bash
1. Parse package.json dependencies
2. Scan for framework-specific config files
3. Check directory structure patterns
4. Assign confidence score (0-100)
5. Select primary framework (highest score)
6. Tag secondary frameworks
```

Phase 04: CI Negotiation

Module: `brain.frameworks.ci.sh`
Purpose: Determine optimal build/test commands
Risk: None (read-only)

Build Command Matrix:

Framework	Build Command	Test Command	Output Dir
Next.js	npm run build	npm test	.next/
Next.js (standalone)	npm run build	npm test	.next/standalone/
Nuxt	npm run build	npm test	.output/
Angular	npm run build -- --configuration=production	npm test -- --watch=false --browsers=ChromeHeadless	dist/
Astro	npm run build	npm test	dist/
Create React App	npm run build	npm test -- --watchAll=false	build/
Vite	npm run build	npm test	dist/
Foundry	forge build	forge test	out/
Hardhat	npx hardhat compile	npx hardhat test	artifacts/

Memory Optimization:

```bash
# Node.js Memory Settings
NODE_OPTIONS="--max-old-space-size=4096"

# Next.js Specific
# Prevent OOM during build
output: "standalone"
experimental: {
  outputFileTracingIncludes: {
    '/api/**/*': ['./node_modules/**/*.wasm']
  }
}
```

Phase 05: Web3 Discovery

Module: brain.solidity.detect.sh

Purpose: Identify smart contract toolchains
Risk: None (read-only)

Foundry Detection:

```bash
# Primary Signatures
foundry.toml
lib/forge-std/

# Secondary Signatures
script/*.s.sol
test/*.t.sol

# Version Detection
forge --version
```

Hardhat Detection:

```bash
# Primary Signatures
hardhat.config.js
hardhat.config.ts

# Secondary Signatures
test/*.js
test/*.ts
scripts/deploy.js

# Version Detection
npx hardhat --version
```

Hybrid Project Detection:

```bash
# Both Foundry and Hardhat present
if [[ -f "foundry.toml" ]] && [[ -f "hardhat.config.js" ]]; then
  TOOLCHAIN="hybrid"
  PRIMARY=$(check_recent_usage)
fi
```

Phase 06: Smart Contract CI Contract

Module: brain.solidity.ci.sh
Purpose: Prepare Web3 build/test commands
Risk: None (read-only)

Foundry CI Template:

```yaml
name: Foundry CI

on:
  push:
    branches: [main, develop]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          submodules: recursive

      - name: Install Foundry
        uses: foundry-rs/foundry-toolchain@v1

      - name: Run Forge build
        run: forge build --sizes

      - name: Run Forge tests
        run: forge test -vvv

      - name: Run Forge coverage
        run: forge coverage
```

Hardhat CI Template:

```yaml
```

```yaml
name: Hardhat CI

on:
  push:
    branches: [main, develop]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Compile contracts
        run: npx hardhat compile

      - name: Run tests
        run: npx hardhat test

      - name: Check contract sizes
        run: npx hardhat size-contracts
```

Phase 07: Forge Surgeon

Module: `brain.forge.repair.sh`
Purpose: Standardize Foundry configuration
Risk: Low (modifies remappings.txt)

Repair Operations:

```bash
```

```
# 1. Standardize remappings.txt
@openzeppelin/=lib/openzeppelin-contracts/
forge-std/=lib/forge-std/src/
ds-test/=lib/ds-test/src/

# 2. Ensure submodules initialized
git submodule update --init --recursive

# 3. Verify lib/ dependencies
lib/forge-std/
lib/openzeppelin-contracts/

# 4. Check foundry.toml configuration
[profile.default]
src = "contracts"
out = "out"
libs = ["lib"]
solc = "0.8.20"
optimizer = true
optimizer_runs = 200

# 5. Validate build
forge build --sizes
```

Common Issues Resolved:

```bash
# Missing remappings
ERROR: Could not find '@openzeppelin/contracts/token/ERC20/ERC20.sol'
FIX: Add '@openzeppelin/=lib/openzeppelin-contracts/' to remappings.txt

# Submodule not initialized
ERROR: lib/forge-std/ directory empty
FIX: git submodule update --init --recursive

# Solidity version mismatch
ERROR: Source uses ^0.8.0 but compiler is 0.7.6
FIX: Update solc version in foundry.toml
```

Phase 08: Rust Synchronization

Module: brain.rust.sh
Purpose: Audit Rust toolchain alignment
Risk: None (read-only)

Cargo.toml Analysis:

```toml
[package]
name = "repo-name"
version = "0.1.0"
edition = "2021"

[dependencies]
tokio = { version = "1.35", features = ["full"] }
serde = { version = "1.0", features = ["derive"] }

[dev-dependencies]
criterion = "0.5"

[profile.release]
opt-level = 3
lto = true
```

Toolchain Verification:

```bash
# Check Rust version
rustc --version

# Check Cargo version
cargo --version

# Verify edition compatibility
EDITION=$(grep 'edition' Cargo.toml | cut -d"" -f2)
if [[ "$EDITION" < "2021" ]]; then
  WARN "Consider upgrading to edition 2021"
fi

# Check for outdated dependencies
cargo outdated
```

Common Configurations:

```bash
# Workspace configuration
[workspace]
members = ["crate-a", "crate-b"]
resolver = "2"

# Build optimization
[profile.release]
opt-level = 3
lto = "fat"
codegen-units = 1
strip = true
```

Phase 09: Genome Injection

Module: `brain.normalize.sh`
Purpose: Deploy canonical workflow templates
Risk: Low (creates/updates .github/workflows/)

Injection Strategy:

```bash
```

```
# 1. Detect project type from Phase 03
PROJECT_TYPE=$(cat diagnosis.json | jq -r '.framework.primary')

# 2. Select canonical template
case "$PROJECT_TYPE" in
  "next")
    TEMPLATE="next-ci.yml"
    ;;
  "angular")
    TEMPLATE="angular-ci.yml"
    ;;
  "solidity")
    TEMPLATE="solidity-ci.yml"
    ;;
esac

# 3. Check for conflicts with Phase 02 audit
CONFLICTS=$(check_workflow_conflicts "$TEMPLATE")

# 4. Inject or update workflow
if [[ -z "$CONFLICTS" ]]; then
  cp "repo-brain/templates/$TEMPLATE" ".github/workflows/$TEMPLATE"
else
  WARN "Workflow conflict detected: $CONFLICTS"
  # Create alternative workflow name
fi
```

Template Customization:

```yaml
```

```yaml
# next-ci.yml base template
name: Next.js CI

on:
  push:
    branches: [$DEFAULT_BRANCH]
  pull_request:

env:
  NODE_VERSION: $DETECTED_NODE_VERSION

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: ${{ env.NODE_VERSION }}
          cache: $DETECTED_PACKAGE_MANAGER

      - name: Install dependencies
        run: $INSTALL_COMMAND

      - name: Build
        run: $BUILD_COMMAND
        env:
          NODE_OPTIONS: --max-old-space-size=4096
```

Phase 10: Deployment Specialist

Module: `brain.vercel.troubleshoot.sh`
Purpose: Detect and fix Vercel OOM issues
Risk: Low (modifies next.config.js)

OOM Signature Detection:

```bash
```

```
# Build log patterns
"JavaScript heap out of memory"
"FATAL ERROR: Reached heap limit"
"ENOSPC: no space left on device"

# Memory usage analysis
BUILD_SIZE=$(du -sh .next/ | cut -f1)
if [[ "$BUILD_SIZE" > "500M" ]]; then
  FLAG_OOM_RISK=true
fi
```

Standalone Mode Enforcement:

```javascript
// next.config.js repair
module.exports = {
  output: 'standalone',
  experimental: {
    outputFileTracingIncludes: {
      '/api/**/*': ['./node_modules/**/*.wasm'],
    },
  },
  webpack: (config, { isServer }) => {
    if (!isServer) {
      config.optimization.minimize = true;
    }
    return config;
  },
}
```

Vercel Configuration:

```json
```

```
// vercel.json
{
  "builds": [
    {
      "src": "package.json",
      "use": "@vercel/next",
      "config": {
        "maxLambdaSize": "50mb"
      }
    }
  ],
  "functions": {
    "app/**": {
      "memory": 3008,
      "maxDuration": 30
    }
  }
}
```

Phase 11: Health Scoring

Module: `brain.diagnose.sh`
Purpose: Generate comprehensive health report
Risk: None (read-only, creates diagnosis.json)

Scoring Algorithm:

```javascript
```

```
// Base score: 100
let score = 100;

// Deductions
if (!hasPackageJson) score -= 20;
if (!hasGitignore) score -= 10;
if (hasSecurityIssues) score -= 30;
if (buildFails) score -= 40;
if (testsFail) score -= 20;
if (hasMergeConflicts) score -= 25;
if (missingDependencies) score -= 15;
if (outdatedDependencies) score -= 5;
if (noCI) score -= 15;
if (inconsistentFormatting) score -= 5;

// Bonuses
if (hasTests) score += 5;
if (hasTypeScript) score += 5;
if (hasComprehensiveCI) score += 10;
if (hasSecurityScanning) score += 5;

// Clamp to 0-100
score = Math.max(0, Math.min(100, score));
```

State Classification:

```javascript
function classifyState(score, findings) {
  if (score >= 90 && findings.critical.length === 0) {
    return "GREEN";
  } else if (score >= 70 && findings.critical.length === 0) {
    return "YELLOW";
  } else if (findings.critical.length > 0 || score < 50) {
    return "RED";
  } else {
    return "AUTO"; // Can be auto-repaired
  }
}
```

diagnosis.json Schema:

```json
```

```
{
  "version": "3.0",
  "timestamp": "2025-01-27T10:30:00Z",
  "score": 85,
  "state": "YELLOW",
  "languages": ["node", "typescript", "solidity"],
  "framework": {
    "primary": "next",
    "version": "14.0.4",
    "secondary": ["ethers", "wagmi"]
  },
  "findings": {
    "critical": [],
    "high": [
      {
        "type": "security",
        "message": "Potential API key in .env.example",
        "file": ".env.example",
        "line": 12,
        "severity": "high"
      }
    ],
    "medium": [
      {
        "type": "build",
        "message": "Next.js not in standalone mode",
        "file": "next.config.js",
        "autofix": "brain.vercel.troubleshoot.sh"
      }
    ],
    "low": [
      {
        "type": "hygiene",
        "message": "node_modules not in .gitignore",
        "file": ".gitignore",
        "autofix": "brain.fix.safe.sh"
      }
    ]
  },
  "metrics": {
    "files": 1247,
    "loc": 45678,
    "dependencies": {
```

```
    "prod": 42,
    "dev": 18,
    "outdated": 3
  },
  "build_time": 124.5,
  "test_coverage": 78.2
},
"recommendations": [
  "Enable standalone output mode in next.config.js",
  "Update outdated dependencies: react@17.0.2 -> 18.2.0",
  "Add .env to .gitignore"
]
}
```

Phase 12: Non-Invasive Patching

Module: brain.fix.safe.sh
Purpose: Apply low-risk corrections
Risk: Low (safe file modifications)

Safe Operations:

```bash
```

```
# 1. .gitignore Standardization
cat >> .gitignore << EOF

# Repo Doctor additions
node_modules/
.next/
out/
dist/
build/
.env.local
.env.*.local
.DS_Store
*.log
.repo-brain.lock
diagnosis.json
EOF

# 2. Directory Permissions
chmod +x repo-brain/*.sh

# 3. Line Ending Normalization
git config core.autocrlf input
find . -type f -name "*.sh" -exec dos2unix {} \;

# 4. Package Manager Consistency
if [[ -f "package-lock.json" ]] && [[ -f "yarn.lock" ]]; then
  WARN "Both npm and yarn lock files present"
  # Keep most recently modified
fi

# 5. README Generation
if [[ ! -f "README.md" ]]; then
  cat > README.md << EOF
# $(basename $(pwd))

This repository is managed by Repo Doctor.

## Build
\`\`\`bash
npm install
npm run build
\`\`\`
```

```
## Test
\`\`\`bash
npm test
\`\`\`
EOF
fi
```

Never Modified:

- Source code files (*.js, *.ts, *.sol, *.rs)
- Configuration with complex logic (webpack, vite)
- Database schemas or migrations
- API contracts or OpenAPI specs

Phase 13: Motor Function Check

Module: `brain.verify.sh`
Purpose: Execute build/test contracts
Risk: None (read-only execution)

Verification Sequence:

```
bash
```

```
# 1. Install Dependencies
if [[ -f "package.json" ]]; then
  npm ci --prefer-offline --no-audit
fi

if [[ -f "Cargo.toml" ]]; then
  cargo fetch
fi

# 2. Run Build
BUILD_COMMAND=$(cat diagnosis.json | jq -r '.ci.build_command')
eval "$BUILD_COMMAND" 2>&1 | tee build.log

BUILD_EXIT_CODE=$?

# 3. Run Tests
TEST_COMMAND=$(cat diagnosis.json | jq -r '.ci.test_command')
eval "$TEST_COMMAND" 2>&1 | tee test.log

TEST_EXIT_CODE=$?

# 4. Check for Merge Conflicts
CONFLICT_MARKERS=$(grep -r "^<<<<<<< HEAD" --include="*.js" --include="*.ts" --include="*.sol" .)
if [[ -n "$CONFLICT_MARKERS" ]]; then
  ERROR "Merge conflict markers detected"
  exit 1
fi

# 5. Verify Output Artifacts
OUTPUT_DIR=$(cat diagnosis.json | jq -r '.ci.output_dir')
if [[ ! -d "$OUTPUT_DIR" ]]; then
  ERROR "Build output directory not found: $OUTPUT_DIR"
  exit 1
fi

# 6. Record Metrics
BUILD_SIZE=$(du -sh "$OUTPUT_DIR" | cut -f1)
echo "Build size: $BUILD_SIZE"

# 7. Exit Status
if [[ $BUILD_EXIT_CODE -eq 0 ]] && [[ $TEST_EXIT_CODE -eq 0 ]]; then
  SUCCESS "Motor function check passed"
  exit 0
```

```
else
  ERROR "Motor function check failed"
  exit 1
fi
```

Phase 14: Forensic Security

Module: `brain.ai.guard.sh`
Purpose: Deep scan for security vulnerabilities
Risk: None (read-only)

Security Patterns:

```bash
```

```
# API Key Patterns
OPENAI_PATTERN='sk-[a-zA-Z0-9]{48}'
ANTHROPIC_PATTERN='sk-ant-[a-zA-Z0-9\-]{95}'
GEMINI_PATTERN='AIza[0-9A-Za-z\-_]{35}'
AWS_PATTERN='AKIA[0-9A-Z]{16}'
STRIPE_PATTERN='sk_live_[0-9a-zA-Z]{24}'

# Private Key Patterns
ETH_PRIVATE_KEY='0x[a-fA-F0-9]{64}'
PEM_PRIVATE_KEY='-----BEGIN (RSA |EC |)PRIVATE KEY-----'

# Injection Patterns
EVAL_PATTERN='eval\s*\('
EXEC_PATTERN='exec\s*\('
SYSTEM_PATTERN='os\.system\s*\('
SHELL_EXEC='shell_exec\s*\('
CHILD_PROCESS='child_process\.exec\('

# SQL Injection
SQL_CONCAT='\+\s*["\']SELECT|INSERT|UPDATE|DELETE'
SQL_TEMPLATE='`SELECT.*\$\{.*\}`'

# Path Traversal
PATH_TRAVERSAL='\.\.\/|\.\.\\|%2e%2e%2f'

# Hardcoded Credentials
PASSWORD_PATTERN='password\s*=\s*["\'][^"\']+["\']'
TOKEN_PATTERN='token\s*=\s*["\'][^"\']+["\']'
```

Scan Execution:

```bash
```

```
# Exclude safe directories
EXCLUDE_DIRS=(
  "node_modules"
  ".git"
  "dist"
  "build"
  ".next"
  "coverage"
)

# Scan all source files
find . -type f \
  \( -name "*.js" -o -name "*.ts" -o -name "*.jsx" -o -name "*.tsx" \
  -o -name "*.py" -o -name "*.sol" -o -name "*.rs" \
  -o -name "*.env*" -o -name "*.config.js" \) \
  $(printf "! -path '*/%s/*' " "${EXCLUDE_DIRS[@]}") \
  -exec grep -H -n -E "$COMBINED_PATTERN" {} \;
```

Advisory Generation:

```
json
```

```
{
  "security_findings": [
    {
      "severity": "CRITICAL",
      "type": "secret_leak",
      "pattern": "OpenAI API Key",
      "file": "src/config.ts",
      "line": 42,
      "snippet": "const apiKey = 'sk-...'",
      "recommendation": "Move to environment variable, add .env to .gitignore",
      "cve": null
    },
    {
      "severity": "HIGH",
      "type": "code_injection",
      "pattern": "eval()",
      "file": "src/utils/parser.js",
      "line": 88,
      "snippet": "eval(userInput)",
      "recommendation": "Replace with JSON.parse or safe alternative",
      "cve": "CWE-95"
    }
  ]
}
```

Phase 15: Integrity Sealing

Module: brain.greenlock.sh
Purpose: Apply SHA-256 genome lock
Risk: None (creates .repo-brain.lock)

Lock Generation:

```bash
```

```bash
# 1. Collect genome files
GENOME_FILES=(
  "repo-brain/*.sh"
  ".github/workflows/brain*.yml"
  "diagnosis.json"
)

# 2. Generate SHA-256 hashes
cat > .repo-brain.lock << EOF
# Repo Doctor Genome Lock
# Generated: $(date -u +"%Y-%m-%dT%H:%M:%SZ")
# State: GREEN

EOF

for file in "${GENOME_FILES[@]}"; do
  if [[ -f "$file" ]]; then
    HASH=$(sha256sum "$file" | cut -d' ' -f1)
    echo "$HASH  $file" >> .repo-brain.lock
  fi
done

# 3. Sign lock file
SIGNATURE=$(sha256sum .repo-brain.lock | cut -d' ' -f1)
echo "" >> .repo-brain.lock
echo "# Lock Signature: $SIGNATURE" >> .repo-brain.lock
```

Lock Verification:

```bash
```

```bash
# Verify genome integrity
while IFS= read -r line; do
  if [[ "$line" =~ ^([a-f0-9]{64})\ \ (.+)$ ]]; then
    EXPECTED_HASH="${BASH_REMATCH[1]}"
    FILE="${BASH_REMATCH[2]}"

    if [[ -f "$FILE" ]]; then
      ACTUAL_HASH=$(sha256sum "$FILE" | cut -d' ' -f1)
      if [[ "$EXPECTED_HASH" != "$ACTUAL_HASH" ]]; then
        ERROR "Genome integrity violation: $FILE"
        exit 1
      fi
    else
      WARN "Genome file missing: $FILE"
    fi
  fi
done < .repo-brain.lock
```

Green Lock Criteria:

```bash
# Repository must meet ALL criteria:
1. diagnosis.json state == "GREEN"
2. All security scans pass (Phase 14)
3. Build and test succeed (Phase 13)
4. No critical or high findings
5. No merge conflicts
6. All genome files present and executable
7. CI workflows operational
```

Phase 16: Neural Contract

Module: `brain.guard.sh`
Purpose: Generate AI governance instructions
Risk: None (creates .github/copilot-instructions.md)

Contract Template:

```markdown
```

AI Assistant Governance Contract

Repository State: GREEN
Score: 92/100
Last Updated: 2025-01-27T10:30:00Z
Managed by: Repo Doctor v3.0

Architecture Overview

This is a **Next.js 14** application with **Solidity** smart contracts.

Stack
- Frontend: Next.js 14, React 18, TypeScript
- Smart Contracts: Solidity 0.8.20, Foundry
- Web3: ethers.js v6, wagmi
- Styling: Tailwind CSS

Directory Structure

```
src/
app/          # Next.js App Router
components/   # React components
lib/          # Utility functions
contracts/      # Solidity smart contracts
lib/          # External contract dependencies
test/           # Contract tests
```

Critical Rules

DO NOT MODIFY
1. `foundry.toml` - Standardized by Repo Doctor
2. `remappings.txt` - Auto-generated dependency mappings
3. `repo-brain/` - Brain module directory
4. `.github/workflows/brain*.yml` - Governance workflows

ALWAYS FOLLOW
1. Use TypeScript for all new files
2. Follow existing import patterns
3. Run `npm run lint` before committing
4. Add tests for new contract functions
5. Update comments for complex logic

SECURITY REQUIREMENTS
1. Never hardcode API keys or secrets
2. Validate all user inputs
3. Use environment variables for config
4. Follow Solidity best practices (CEI pattern)
5. Run security scan before deploying contracts

Build Commands
```bash
# Frontend
npm install
npm run build

# Smart Contracts
forge install
forge build
forge test
```

Testing
```bash
# Frontend tests
npm test

# Contract tests
forge test -vvv
```

Coverage
forge coverage
```

## Current Health Status

### ✅ Strengths
- Comprehensive test coverage (78%)
- No critical security issues
- Optimized build pipeline
- Automated CI/CD

### ⚠️ Warnings
- 3 dependencies outdated
- Next.js standalone mode recommended

### 📋 TODO
- Update react@17 to react@18
- Add E2E tests
- Implement contract size monitoring

## Repo Doctor Commands
```bash
# Run full health check
./repo-brain/brainctl.sh diagnose

# Repair Foundry configuration
./repo-brain/brainctl.sh forge-repair

# View repository vitals
./repo-brain/brainctl.sh vitals

# Generate forensic trace
./repo-brain/brainctl.sh autopsy
```

Deployment

- **Frontend:** Vercel (main branch auto-deploys)
- **Contracts:** Manual deployment via Foundry scripts

```

This contract is automatically updated by Repo Doctor.

For issues, see: diagnosis.json

Phase 17: Auto-Wiring

Module: `brain.auto-pr.sh`

Purpose: Create autonomous repair PRs

Risk: Medium (creates branches, commits, pushes)

PR Creation Flow:

```bash
```

```
# 1. Check if repairs are available
REPAIRS=$(cat diagnosis.json | jq -r '.findings.medium[] | select(.autofix != null)')

if [[ -z "$REPAIRS" ]]; then
  INFO "No auto-repairs available"
  exit 0
fi

# 2. Create repair branch
BRANCH_NAME="repo-doctor/auto-repair-$(date +%Y%m%d-%H%M%S)"
git checkout -b "$BRANCH_NAME"

# 3. Apply repairs
while IFS= read -r repair; do
  AUTOFIX_SCRIPT=$(echo "$repair" | jq -r '.autofix')
  INFO "Running: $AUTOFIX_SCRIPT"
  bash "repo-brain/$AUTOFIX_SCRIPT"
done <<< "$REPAIRS"

# 4. Commit changes
git add -A
git commit -m "🏥 Repo Doctor: Automated infrastructure repair

Applied fixes for:
$(echo "$REPAIRS" | jq -r '.message' | sed 's/^/- /')

Generated by: brain.auto-pr.sh
Diagnosis: See diagnosis.json"

# 5. Push to origin
git push origin "$BRANCH_NAME"

# 6. Create PR via GitHub CLI
gh pr create \
  --title "🏥 Repo Doctor: Automated Infrastructure Repair" \
  --body "$(generate_pr_body)" \
  --label "repo-doctor,automated,infrastructure" \
  --assignee "@me"
```

PR Body Template:

```markdown
```

🏥 Automated Repair by Repo Doctor

This PR contains automated infrastructure repairs identified during the latest health check.

📊 Repository Health
- **Previous Score:** 78/100
- **Expected Score:** 85/100
- **State:** YELLOW → GREEN

🔧 Repairs Applied
- ✅ Added missing dependencies to .gitignore
- ✅ Enabled Next.js standalone output mode
- ✅ Standardized Foundry remappings
- ✅ Fixed directory permissions

📋 Changes
File	Change Type	Description
`.gitignore`	Modified	Added node_modules, .env.local
`next.config.js`	Modified	Added standalone output
`remappings.txt`	Created	Standardized dependency paths
`repo-brain/*.sh`	Permissions	Made all scripts executable

✅ Verification
- [x] Build succeeds locally
- [x] Tests pass
- [x] No merge conflicts
- [x] Security scan clean

🤖 Automation Details
- **Generated by:** brain.auto-pr.sh (Phase 17)
- **Triggered by:** GitHub Actions workflow
- **Diagnosis:** [View diagnosis.json](./diagnosis.json)

📝 Review Checklist
- [ ] Changes align with project conventions
- [ ] No unintended side effects
- [ ] All CI checks pass
- [ ] Ready to merge

This PR was automatically created by Repo Doctor.

For questions, see the [Repo Doctor documentation](./repo-brain/README.md).

Phase 18: Telemetry Sync

Module: brain.fleet.sh
Purpose: Update fleet-level metrics
Risk: None (updates brain.fleet.json)

Fleet Telemetry Schema:

```
json
```

```
{
  "fleet_version": "3.0",
  "last_sync": "2025-01-27T10:30:00Z",
  "repositories": [
    {
      "name": "frontend-app",
      "path": "./frontend-app",
      "state": "GREEN",
      "score": 92,
      "languages": ["node", "typescript"],
      "framework": "next",
      "last_diagnosis": "2025-01-27T09:15:00Z",
      "build_time": 124.5,
      "test_coverage": 78.2,
      "dependencies": {
        "total": 60,
        "outdated": 3,
        "vulnerable": 0
      },
      "issues": {
        "critical": 0,
        "high": 0,
        "medium": 2,
        "low": 5
      }
    },
    {
      "name": "smart-contracts",
      "path": "./smart-contracts",
      "state": "YELLOW",
      "score": 81,
      "languages": ["solidity"],
      "framework": "foundry",
      "last_diagnosis": "2025-01-27T09:20:00Z",
      "build_time": 45.2,
      "test_coverage": 92.5,
      "contract_sizes": {
        "Token": "12.4 KB",
        "Staking": "18.7 KB"
      },
      "issues": {
        "critical": 0,
        "high": 1,
```

```
      "medium": 3,
      "low": 2
    }
  }
],
"fleet_metrics": {
  "total_repos": 2,
  "green": 1,
  "yellow": 1,
  "red": 0,
  "auto": 0,
  "avg_score": 86.5,
  "total_issues": 13,
  "repos_with_ci": 2,
  "repos_with_tests": 2
}
}
```


Sync Operations:

```bash
```

```
# 1. Load existing fleet data
if [[ -f "brain.fleet.json" ]]; then
  FLEET_DATA=$(cat brain.fleet.json)
else
  FLEET_DATA='{
    "fleet_version": "3.0",
    "repositories": []
  }'
fi

# 2. Update current repository entry
CURRENT_DIAGNOSIS=$(cat diagnosis.json)
REPO_NAME=$(basename $(pwd))

FLEET_DATA=$(echo "$FLEET_DATA" | jq \
  --arg name "$REPO_NAME" \
  --argjson diag "$CURRENT_DIAGNOSIS" \
  '
  .last_sync = now | todate |
  .repositories |= map(
    if .name == $name then . + $diag else . end
  ) |
  if (.repositories | map(.name) | index($name)) == null then
    .repositories += [$diag]
  else . end
  ')

# 3. Recalculate fleet metrics
FLEET_DATA=$(echo "$FLEET_DATA" | jq '
  .fleet_metrics = {
    total_repos: (.repositories | length),
    green: (.repositories | map(select(.state == "GREEN")) | length),
    yellow: (.repositories | map(select(.state == "YELLOW")) | length),
    red: (.repositories | map(select(.state == "RED")) | length),
    auto: (.repositories | map(select(.state == "AUTO")) | length),
    avg_score: (.repositories | map(.score) | add / length),
    total_issues: (.repositories | map(.issues | add) | add),
    repos_with_ci: (.repositories | map(select(.ci != null)) | length),
    repos_with_tests: (.repositories | map(select(.test_coverage > 0)) | length)
  }
')
```

```
# 4. Write updated fleet data
echo "$FLEET_DATA" | jq '.' > brain.fleet.json
```

Phase 19: Graduation Suite

Module: brain.test.sh
Purpose: Validate brain modules
Risk: None (read-only testing)

Test Suite:

```
bash
```

```bash
#!/bin/bash
# brain.test.sh - Graduation Suite

set -euo pipefail

TESTS_PASSED=0
TESTS_FAILED=0

# Test 1: All brain modules exist
test_modules_exist() {
  local REQUIRED_MODULES=(
    "brain.run.sh"
    "brain.detect.sh"
    "brain.diagnose.sh"
    "brain.verify.sh"
    "brain.ai.guard.sh"
  )

  for module in "${REQUIRED_MODULES[@]}"; do
    if [[ ! -f "repo-brain/$module" ]]; then
      echo "❌ Missing module: $module"
      return 1
    fi
  done

  echo "✅ All required modules present"
  return 0
}

# Test 2: All modules are executable
test_modules_executable() {
  for script in repo-brain/*.sh; do
    if [[ ! -x "$script" ]]; then
      echo "❌ Not executable: $script"
      return 1
    fi
  done

  echo "✅ All modules executable"
  return 0
}

# Test 3: Syntax check all scripts
```

```
test_syntax() {
  for script in repo-brain/*.sh; do
    if ! bash -n "$script" 2>/dev/null; then
      echo "❌ Syntax error in: $script"
      return 1
    fi
  done

  echo "✅ All scripts have valid syntax"
  return 0
}

# Test 4: Detect module produces valid JSON
test_detect_output() {
  local OUTPUT=$(bash repo-brain/brain.detect.sh)

  if ! echo "$OUTPUT" | jq '.' >/dev/null 2>&1; then
    echo "❌ brain.detect.sh did not produce valid JSON"
    return 1
  fi

  echo "✅ Detection module produces valid JSON"
  return 0
}

# Test 5: Diagnose module creates diagnosis.json
test_diagnose_output() {
  bash repo-brain/brain.diagnose.sh >/dev/null 2>&1

  if [[ ! -f "diagnosis.json" ]]; then
    echo "❌ diagnosis.json not created"
    return 1
  fi

  if ! jq '.' diagnosis.json >/dev/null 2>&1; then
    echo "❌ diagnosis.json is not valid JSON"
    return 1
  fi

  echo "✅ Diagnosis module creates valid output"
  return 0
}

# Test 6: GitHub workflows are valid YAML
```

```
test_workflows_valid() {
  for workflow in .github/workflows/*.yml; do
    if [[ -f "$workflow" ]]; then
      if ! python3 -c "import yaml; yaml.safe_load(open('$workflow'))" 2>/dev/null; then
        echo "❌ Invalid YAML: $workflow"
        return 1
      fi
    fi
  done

  echo "✅ All workflows are valid YAML"
  return 0
}

# Test 7: No shell security anti-patterns
test_security_patterns() {
  local PATTERNS=(
    "eval.*\$"
    "rm -rf \$"
    "\$\(.*curl.*\)"
  )

  for pattern in "${PATTERNS[@]}"; do
    if grep -r -E "$pattern" repo-brain/*.sh >/dev/null 2>&1; then
      echo "❌ Dangerous pattern found: $pattern"
      return 1
    fi
  done

  echo "✅ No security anti-patterns detected"
  return 0
}

# Run all tests
run_tests() {
  local TESTS=(
    "test_modules_exist"
    "test_modules_executable"
    "test_syntax"
    "test_detect_output"
    "test_diagnose_output"
    "test_workflows_valid"
    "test_security_patterns"
  )
```

```
  echo "🧪 Running Graduation Suite..."
  echo ""

  for test in "${TESTS[@]}"; do
    if $test; then
      ((TESTS_PASSED++))
    else
      ((TESTS_FAILED++))
    fi
    echo ""
  done

  echo "━━━
```

```yaml
name: 🏥 Repo Doctor - Master Admission

on:
  push:
    branches: [main, master, develop]
  pull_request:
  workflow_dispatch:
  schedule:
    - cron: '0 0 * * 0' # Weekly health check

env:
  NODE_VERSION: '20'
  FOUNDRY_VERSION: 'nightly'

jobs:
  admission:
    name: Hospital Admission
    runs-on: ubuntu-latest
    permissions:
      contents: write
      pull-requests: write
      issues: write

    steps:
      - name: 📥 Checkout Repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0
          submodules: recursive

      - name: 🔧 Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: ${{ env.NODE_VERSION }}
          cache: 'npm'

      - name: ⚡ Setup Foundry
        uses: foundry-rs/foundry-toolchain@v1
        with:
          version: ${{ env.FOUNDRY_VERSION }}

      - name: 🦀 Setup Rust
        uses: dtolnay/rust-toolchain@stable
```

```yaml
- name: 🐍 Setup Python
  uses: actions/setup-python@v5
  with:
    python-version: '3.11'

- name: 🧠 Make Brain Modules Executable
  run: chmod +x repo-brain/*.sh

- name: 🏥 Run Full Admission Pipeline
  id: admission
  run: |
    set +e
    bash repo-brain/brain.run.sh
    EXIT_CODE=$?
    set -e

    echo "exit_code=$EXIT_CODE" >> $GITHUB_OUTPUT

    if [[ -f diagnosis.json ]]; then
      STATE=$(jq -r '.state' diagnosis.json)
      SCORE=$(jq -r '.score' diagnosis.json)
      echo "state=$STATE" >> $GITHUB_OUTPUT
      echo "score=$SCORE" >> $GITHUB_OUTPUT
    fi

- name: 📊 Upload Diagnosis
  uses: actions/upload-artifact@v4
  if: always()
  with:
    name: diagnosis
    path: |
      diagnosis.json
      build.log
      test.log

- name: 🔒 Verify Integrity Lock
  if: steps.admission.outputs.state == 'GREEN'
  run: |
    if [[ -f .repo-brain.lock ]]; then
      echo "✅ Green lock applied"
    else
      echo "⚠️ Green lock missing"
      exit 1
```

```
        fi

    - name: 📝 Post AI Security Advisory
      if: github.event_name == 'pull_request'
      uses: actions/github-script@v7
      with:
        script: |
          const fs = require('fs');
          const diagnosis = JSON.parse(fs.readFileSync('diagnosis.json', 'utf8'));

          const securityFindings = diagnosis.findings?.high
            ?.filter(f => f.type === 'security') || [];

          if (securityFindings.length > 0) {
            const body = `## 🛡️ Security Advisory from Repo Doctor

${securityFindings.map(f => `
### ${f.severity}: ${f.message}
- **File:** \`${f.file}\`
- **Line:** ${f.line}
- **Recommendation:** ${f.recommendation}
`).join('\n')}

---
*Generated by Repo Doctor AI Guard (Phase 14)*`;

            await github.rest.issues.createComment({
              owner: context.repo.owner,
              repo: context.repo.repo,
              issue_number: context.issue.number,
              body: body
            });
          }

    - name: 🎓 Graduation Check
      if: always()
      run: |
        STATE="${{ steps.admission.outputs.state }}"
        SCORE="${{ steps.admission.outputs.score }}"

        echo "━━━
```

```
echo "Score: $SCORE/100"
echo "━━━
```

```yaml
name: 🩺 Brain Self-Healing

on:
  schedule:
    - cron: '0 2 * * *' # Daily at 2 AM
  workflow_dispatch:

jobs:
  self-audit:
    name: Brain Health Check
    runs-on: ubuntu-latest
    permissions:
      contents: write

    steps:
      - uses: actions/checkout@v4

      - name: 🩺 Run Brain Doctor
        id: doctor
        run: |
          chmod +x repo-brain/*.sh
          bash repo-brain/brain.doctor.sh
          echo "health=$?" >> $GITHUB_OUTPUT

      - name: 🔧 Run Brain Surgeon if Needed
        if: steps.doctor.outputs.health != '0'
        run: |
          bash repo-brain/brain.surgeon.sh

          if git diff --quiet; then
            echo "No repairs needed"
          else
            git config user.name "Repo Doctor Bot"
            git config user.email "repo-doctor@cyberai.network"
            git add repo-brain/
            git commit -m "🔧 Brain Surgeon: Restore genomic integrity"
            git push
          fi
```

brain-auto-comments.yml - PR Advisory

Purpose: Post forensic findings to PRs
Triggers: Pull request
Runtime: ~1-2 minutes

```yaml
```

```yaml
name: 🔍 Brain Auto-Comments

on:
  pull_request:
    types: [opened, synchronize]

jobs:
  forensics:
    name: Forensic Analysis
    runs-on: ubuntu-latest
    permissions:
      pull-requests: write

    steps:
      - uses: actions/checkout@v4

      - name: 🔍 Run AI Guard
        id: guard
        run: |
          chmod +x repo-brain/brain.ai.guard.sh
          bash repo-brain/brain.ai.guard.sh > guard_output.json

      - name: 💬 Post Findings
        uses: actions/github-script@v7
        with:
          script: |
            const fs = require('fs');

            if (!fs.existsSync('guard_output.json')) {
              console.log('No security findings');
              return;
            }

            const findings = JSON.parse(fs.readFileSync('guard_output.json'));

            if (findings.security_findings.length === 0) {
              await github.rest.issues.createComment({
                owner: context.repo.owner,
                repo: context.repo.repo,
                issue_number: context.issue.number,
                body: '✅ **Security Scan:** No issues detected'
              });
              return;
```

```
}

const critical = findings.security_findings.filter(f => f.severity === 'CRITICAL');
const high = findings.security_findings.filter(f => f.severity === 'HIGH');

let body = '## 🔍 Repo Doctor Security Analysis\n\n';

if (critical.length > 0) {
  body += '### 🚨 Critical Issues\n\n';
  critical.forEach(f => {
    body += `- **${f.type}** in \`${f.file}:${f.line}\`\n`;
    body += `  ${f.recommendation}\n\n`;
  });
}

if (high.length > 0) {
  body += '### ⚠️ High Priority Issues\n\n';
  high.forEach(f => {
    body += `- **${f.type}** in \`${f.file}:${f.line}\`\n`;
    body += `  ${f.recommendation}\n\n`;
  });
}

body += '\n---\n*Automated by Repo Doctor AI Guard*';

await github.rest.issues.createComment({
  owner: context.repo.owner,
  repo: context.repo.repo,
  issue_number: context.issue.number,
  body: body
});
```

Canonical CI Templates

next-ci.yml

```yaml
```

```yaml
name: Next.js CI

on:
  push:
    branches: [main, develop]
  pull_request:

env:
  NODE_OPTIONS: --max-old-space-size=4096

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install Dependencies
        run: npm ci

      - name: Lint
        run: npm run lint

      - name: Type Check
        run: npm run type-check || true

      - name: Build
        run: npm run build

      - name: Test
        run: npm test -- --passWithNoTests

      - name: Check Build Size
        run: |
          BUILD_SIZE=$(du -sh .next | cut -f1)
          echo "Build size: $BUILD_SIZE"
```

solidity-ci.yml

```yaml
```

```yaml
name: Solidity CI

on:
  push:
    branches: [main, develop]
  pull_request:

jobs:
  foundry:
    runs-on: ubuntu-latest
    if: "contains(github.event.head_commit.message, '[foundry]') || github.event_name == 'pull_request'"

    steps:
      - uses: actions/checkout@v4
        with:
          submodules: recursive

      - name: Install Foundry
        uses: foundry-rs/foundry-toolchain@v1

      - name: Build
        run: forge build --sizes

      - name: Test
        run: forge test -vvv

      - name: Coverage
        run: forge coverage

  hardhat:
    runs-on: ubuntu-latest
    if: "contains(github.event.head_commit.message, '[hardhat]')"

    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install Dependencies
```

```yaml
    run: npm ci

  - name: Compile
    run: npx hardhat compile

  - name: Test
    run: npx hardhat test

  - name: Check Contract Sizes
    run: npx hardhat size-contracts
```

angular-ci.yml

```yaml
```

```yaml
name: Angular CI

on:
  push:
    branches: [main, develop]
  pull_request:

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install Dependencies
        run: npm ci

      - name: Lint
        run: npm run lint

      - name: Build
        run: npm run build -- --configuration=production

      - name: Test
        run: npm test -- --watch=false --browsers=ChromeHeadless --code-coverage

      - name: Upload Coverage
        uses: codecov/codecov-action@v4
        with:
          files: ./coverage/lcov.info
```

Core Shell Modules

brain.run.sh - Master Orchestrator

Purpose: Execute all 19 phases sequentially
Entry Point: Called by brain.yml

```bash
```

```bash
#!/bin/bash
# brain.run.sh - Master Orchestrator

set -euo pipefail

REPO_ROOT=$(pwd)
BRAIN_DIR="$REPO_ROOT/repo-brain"

# Color codes
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
BLUE='\033[0;34m'
NC='\033[0m'

log() {
  echo -e "${BLUE}[$(date +'%Y-%m-%d %H:%M:%S')]${NC} $1"
}

success() {
  echo -e "${GREEN} ✅ $1${NC}"
}

warn() {
  echo -e "${YELLOW} ⚠️  $1${NC}"
}

error() {
  echo -e "${RED} ❌ $1${NC}"
}

# Phase execution tracker
PHASE_NUM=0
PHASES_PASSED=0
PHASES_FAILED=0

run_phase() {
  local PHASE_NAME=$1
  local PHASE_SCRIPT=$2
  local PHASE_REQUIRED=${3:-true}

  ((PHASE_NUM++))
```

```
  log "━━━
```

```
run_phase "CI/CD Audit" "brain.scan-actions.sh"
run_phase "Framework Detection" "brain.frameworks.sh"
run_phase "CI Negotiation" "brain.frameworks.ci.sh"
run_phase "Web3 Discovery" "brain.solidity.detect.sh" false
run_phase "Smart Contract CI" "brain.solidity.ci.sh" false
run_phase "Forge Surgeon" "brain.forge.repair.sh" false
run_phase "Rust Synchronization" "brain.rust.sh" false

# Phase 09-13: Normalization Layer
run_phase "Genome Injection" "brain.normalize.sh"
run_phase "Deployment Specialist" "brain.vercel.troubleshoot.sh" false
run_phase "Health Scoring" "brain.diagnose.sh"
run_phase "Non-Invasive Patching" "brain.fix.safe.sh"
run_phase "Motor Function Check" "brain.verify.sh" false

# Phase 14-16: Security Layer
run_phase "Forensic Security" "brain.ai.guard.sh"
run_phase "Integrity Sealing" "brain.greenlock.sh"
run_phase "Neural Contract" "brain.guard.sh"

# Phase 17-19: Autonomy Layer
run_phase "Auto-Wiring" "brain.auto-pr.sh" false
run_phase "Telemetry Sync" "brain.fleet.sh"
run_phase "Graduation Suite" "brain.test.sh"

# Final report
log "━━━
```

```
    exit 1
  fi

  success "Repository admission successful"
  exit 0
}

# Execute
main
```

---

brainctl.sh - CLI Interface

Purpose: Human-to-brain command interface
Usage: `./repo-brain/brainctl.sh <command>`

```bash
```

```bash
#!/bin/bash
# brainctl.sh - Repo Doctor CLI

set -euo pipefail

BRAIN_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
REPO_ROOT="$(cd "$BRAIN_DIR/.." && pwd)"

# Commands
cmd_help() {
  cat << EOF
🩺 Repo Doctor CLI (brainctl)

USAGE:
  brainctl <command> [options]

COMMANDS:
  run            Run full 19-phase admission pipeline
  diagnose       Generate health report only
  forge-repair   Repair Foundry configuration
  verify         Run build and test
  guard          Run security scan
  autopsy        Generate forensic trace
  vitals         Show repository metrics
  test           Run graduation suite
  lock           Generate integrity lock
  unlock         Remove integrity lock
  fleet          Update fleet telemetry
  doctor         Check brain health
  surgeon        Repair brain modules
  version        Show version info

EXAMPLES:
  brainctl run
  brainctl diagnose
  brainctl forge-repair
  brainctl autopsy --last-build

For more info: https://github.com/cyberai-network/repo-doctor
EOF
}

cmd_run() {
```

```
  echo "🏥 Running full admission pipeline..."
  bash "$BRAIN_DIR/brain.run.sh"
}

cmd_diagnose() {
  echo "🔍 Generating health report..."
  bash "$BRAIN_DIR/brain.detect.sh"
  bash "$BRAIN_DIR/brain.diagnose.sh"

  if [[ -f "$REPO_ROOT/diagnosis.json" ]]; then
    cat "$REPO_ROOT/diagnosis.json" | jq '.'
  fi
}

cmd_forge_repair() {
  echo "🔧 Repairing Foundry configuration..."
  bash "$BRAIN_DIR/brain.forge.repair.sh"
}

cmd_verify() {
  echo "✅ Running motor function check..."
  bash "$BRAIN_DIR/brain.verify.sh"
}

cmd_guard() {
  echo "🛡️ Running security scan..."
  bash "$BRAIN_DIR/brain.ai.guard.sh"
}

cmd_autopsy() {
  echo "🔬 Generating forensic trace..."
  bash "$BRAIN_DIR/brain.autopsy.sh" "$@"
}

cmd_vitals() {
  echo "📊 Collecting repository vitals..."
  bash "$BRAIN_DIR/brain.vitals.sh"
}

cmd_test() {
  echo "🧪 Running graduation suite..."
  bash "$BRAIN_DIR/brain.test.sh"
}
```

```
cmd_lock() {
  echo "🔒 Generating integrity lock..."
  bash "$BRAIN_DIR/brain.greenlock.sh"
}

cmd_unlock() {
  echo "🔓 Removing integrity lock..."
  rm -f "$REPO_ROOT/.repo-brain.lock"
  echo "✅ Lock removed"
}

cmd_fleet() {
  echo "📡 Updating fleet telemetry..."
  bash "$BRAIN_DIR/brain.fleet.sh"
}

cmd_doctor() {
  echo "🩺 Checking brain health..."
  bash "$BRAIN_DIR/brain.doctor.sh"
}

cmd_surgeon() {
  echo "⚕️ Repairing brain modules..."
  bash "$BRAIN_DIR/brain.surgeon.sh"
}

cmd_version() {
  cat << EOF
Repo Doctor v3.0
MERMEDA Protocol
Built by CyberAI Oracle Network
EOF
}

# Main
case "${1:-help}" in
  run) cmd_run ;;
  diagnose) cmd_diagnose ;;
  forge-repair) cmd_forge_repair ;;
  verify) cmd_verify ;;
  guard) cmd_guard ;;
  autopsy) shift; cmd_autopsy "$@" ;;
  vitals) cmd_vitals ;;
  test) cmd_test ;;
```

```
  lock) cmd_lock ;;
  unlock) cmd_unlock ;;
  fleet) cmd_fleet ;;
  doctor) cmd_doctor ;;
  surgeon) cmd_surgeon ;;
  version) cmd_version ;;
  help|--help|-h) cmd_help ;;
  *)
    echo "❌ Unknown command: $1"
    echo "Run 'brainctl help' for usage"
    exit 1
    ;;
esac
```

Governance Rules

1. Immutability Rules

DO NOT MODIFY:

- `repo-brain/` directory structure
- `.github/workflows/brain*.yml` files
- `diagnosis.json` schema
- `brain.fleet.json` structure
- `.repo-brain.lock` once GREEN state achieved

EXCEPTIONS:

- Approved version upgrades
- Security patches
- Explicit manual override with `--force` flag

---

2. Execution Rules

Idempotency:

- All phases MUST be repeatable without side effects
- Running twice should produce identical results
- File modifications must check current state first

Determinism:

- Same input always produces same output
- No random values in core logic
- Timestamps only for metadata

Atomicity:

- Each phase is independent
- Failure in one phase doesn't corrupt others
- Rollback capability for structural changes

---

3. Security Rules

Secret Management:

- Never log API keys or tokens
- Mask secrets in output
- Use environment variables only
- Scan for leaks in every PR

Code Execution:

- No arbitrary `eval()` or `exec()`
- Validate all user inputs
- Sanitize file paths
- Restrict network access

Permissions:

- Least privilege principle
- No sudo or root requirements
- Read-only by default
- Write only when necessary

---

4. Fleet Rules

Synchronization:

- Fleet data updates after every admission
- No cross-repo data pollution
- Independent repository states
- Aggregated metrics only at fleet level

Consistency:

- All repos in fleet use same brain version
- Canonical templates are identical
- Security rules are uniform

---

5. AI Governance Rules

Copilot Instructions:

- Must reflect repository health
- Updated after diagnosis changes
- Include architecture context
- List forbidden modifications
- Specify security requirements

Behavioral Contracts:

- No automatic secret commits
- Respect .gitignore patterns
- Follow existing code style
- Add tests for new features

Codebooks

Codebook A: Genome Rules

Language Signatures:

yaml

```yaml
node:
  primary_indicators:
    - package.json
    - node_modules/
  secondary_indicators:
    - .nvmrc
    - .node-version
    - package-lock.json
  confidence_weight: 1.0

typescript:
  primary_indicators:
    - tsconfig.json
  secondary_indicators:
    - "*.ts"
    - "*.tsx"
  confidence_weight: 0.9

python:
  primary_indicators:
    - setup.py
    - pyproject.toml
    - requirements.txt
  secondary_indicators:
    - "*.py"
    - venv/
  confidence_weight: 1.0

rust:
  primary_indicators:
    - Cargo.toml
  secondary_indicators:
    - src/main.rs
    - src/lib.rs
  confidence_weight: 1.0

solidity:
  primary_indicators:
    - contracts/*.sol
  secondary_indicators:
    - foundry.toml
```

```yaml
    - hardhat.config.js
  confidence_weight: 1.0
```

Codebook B: CI Contracts

Build Command Mappings:

```yaml
next:
  install: npm ci
  build: npm run build
  test: npm test
  output: .next/
  env:
    NODE_OPTIONS: --max-old-space-size=4096

angular:
  install: npm ci
  build: npm run build -- --configuration=production
  test: npm test -- --watch=false --browsers=ChromeHeadless
  output: dist/

foundry:
  install: git submodule update --init --recursive
  build: forge build
  test: forge test
  output: out/

hardhat:
  install: npm ci
  build: npx hardhat compile
  test: npx hardhat test
  output: artifacts/
```

Codebook C: Security Signatures

Patterns to Detect:

```yaml
```

```yaml
api_keys:
  - pattern: 'sk-[a-zA-Z0-9]{48}'
    name: OpenAI API Key
    severity: CRITICAL

  - pattern: 'sk-ant-[a-zA-Z0-9\-]{95}'
    name: Anthropic API Key
    severity: CRITICAL

  - pattern: 'AKIA[0-9A-Z]{16}'
    name: AWS Access Key
    severity: CRITICAL

private_keys:
  - pattern: '-----BEGIN (RSA |EC |)PRIVATE KEY-----'
    name: PEM Private Key
    severity: CRITICAL

  - pattern: '0x[a-fA-F0-9]{64}'
    name: Ethereum Private Key
    severity: CRITICAL

code_injection:
  - pattern: 'eval\s*\('
    name: JavaScript eval()
    severity: HIGH

  - pattern: 'exec\s*\('
    name: Python exec()
    severity: HIGH

  - pattern: 'os\.system\s*\('
    name: OS command injection
    severity: HIGH

sql_injection:
  - pattern: '\+\s*["'']SELECT|INSERT|UPDATE|DELETE'
    name: SQL concatenation
    severity: HIGH

  - pattern: '`SELECT.*\$\{.*\}`'
```

```
name: Template SQL injection
severity: HIGH
```

Codebook D: Repair Policies

Safe Repairs (Low Risk):

- .gitignore additions
- File permission corrections
- README generation
- Package manager lock file cleanup
- Line ending normalization

Structural Repairs (Medium Risk):

- CI workflow injection
- Directory reorganization
- Dependency updates
- Configuration file modifications
- Build script generation

Deep Repairs (High Risk):

- Source code refactoring
- Database schema changes
- API contract modifications
- Architecture restructuring
- Framework upgrades

Repair Decision Matrix:

```
Risk × Impact → Action
──────────────────────────────────────────
Low × Low → Auto-apply
Low × Medium → Auto-apply with PR
Low × High → Auto-apply with review
Medium × Low → Auto-PR
Medium × Medium → Auto-PR with review
Medium × High → Advisory only
High × Any → Advisory only
```

Codebook E: Fleet Governance

Telemetry Schema:

```typescript
```

```typescript
interface FleetTelemetry {
  fleet_version: string;
  last_sync: string; // ISO 8601
  repositories: Repository[];
  fleet_metrics: FleetMetrics;
}

interface Repository {
  name: string;
  path: string;
  state: 'GREEN' | 'YELLOW' | 'RED' | 'AUTO';
  score: number; // 0-100
  languages: string[];
  framework: string;
  last_diagnosis: string;
  issues: IssueCounts;
}

interface FleetMetrics {
  total_repos: number;
  green: number;
  yellow: number;
  red: number;
  auto: number;
  avg_score: number;
  total_issues: number;
}
```

Drift Thresholds:

```yaml
warning_thresholds:
  score_drop: 10 # points
  new_critical_issues: 1
  build_time_increase: 50 # percent

critical_thresholds:
  score_drop: 20 # points
  state_degradation: GREEN → YELLOW
  new_high_issues: 3
```

Addons & Enhancements

Web3 Enhancements

ABI Drift Detection

Module: brain.abi.drift.sh

```bash
# Detect changes in contract ABIs
if [[ -f "artifacts/" ]] && [[ -f ".abi.baseline" ]]; then
  CURRENT_ABI=$(find artifacts/ -name "*.json" -exec cat {} \; | jq '.abi' | sha256sum)
  BASELINE_ABI=$(cat .abi.baseline)

  if [[ "$CURRENT_ABI" != "$BASELINE_ABI" ]]; then
    WARN "ABI drift detected - breaking changes may exist"
    # Generate ABI diff report
  fi
fi
```

Contract Size Regression

Module: brain.contract.size.sh

```bash
# Monitor contract deployment sizes
forge build --sizes | tee contract-sizes.txt

while IFS= read -r line; do
  if [[ "$line" =~ ([A-Za-z0-9]+)\.sol.*\|.*([0-9,]+) ]]; then
    CONTRACT="${BASH_REMATCH[1]}"
    SIZE="${BASH_REMATCH[2]//,/}"

    # Ethereum mainnet limit: 24576 bytes
    if [[ $SIZE -gt 24576 ]]; then
      ERROR "$CONTRACT exceeds 24KB limit: ${SIZE} bytes"
    elif [[ $SIZE -gt 20000 ]]; then
      WARN "$CONTRACT approaching size limit: ${SIZE} bytes"
    fi
  fi
done < contract-sizes.txt
```

Foundry/Hardhat Mismatch Resolution

Module: brain.web3.hybrid.sh

```bash
# Detect hybrid projects and reconcile
if [[ -f "foundry.toml" ]] && [[ -f "hardhat.config.js" ]]; then
  # Compare contract directories
  FOUNDRY_CONTRACTS=$(find contracts/ -name "*.sol" | sort)
  HARDHAT_CONTRACTS=$(grep -r "artifacts" hardhat.config.js | wc -l)

  if [[ $HARDHAT_CONTRACTS -gt 0 ]]; then
    # Both toolchains active - ensure consistency
    INFO "Hybrid Foundry + Hardhat detected"

    # Sync compiler versions
    FOUNDRY_SOLC=$(grep "solc =" foundry.toml | cut -d"" -f2)
    HARDHAT_SOLC=$(grep "solidity:" hardhat.config.js | cut -d"" -f2)

    if [[ "$FOUNDRY_SOLC" != "$HARDHAT_SOLC" ]]; then
      WARN "Compiler version mismatch: Foundry=$FOUNDRY_SOLC, Hardhat=$HARDHAT_SOLC"
    fi
  fi
fi
```

Frontend Enhancements

Next.js OOM Heuristics

Module: brain.nextjs.memory.sh

```bash
```

```
# Detect memory-intensive patterns
check_oom_risk() {
  local RISK_SCORE=0

  # Large bundle size
  if [[ -d ".next" ]]; then
    BUNDLE_SIZE=$(du -sm .next | cut -f1)
    if [[ $BUNDLE_SIZE -gt 100 ]]; then
      ((RISK_SCORE += 30))
    fi
  fi

  # Dynamic imports
  DYNAMIC_IMPORTS=$(grep -r "import(" --include="*.js" --include="*.tsx" | wc -l)
  if [[ $DYNAMIC_IMPORTS -gt 50 ]]; then
    ((RISK_SCORE += 20))
  fi

  # Image optimization
  NEXT_CONFIG=$(grep "images:" next.config.js || true)
  if [[ -z "$NEXT_CONFIG" ]]; then
    ((RISK_SCORE += 15))
  fi

  # Server components
  SERVER_COMPONENTS=$(grep -r '"use server"' app/ | wc -l)
  if [[ $SERVER_COMPONENTS -lt 5 ]]; then
    ((RISK_SCORE += 10))
  fi

  echo $RISK_SCORE
}

OOM_RISK=$(check_oom_risk)
if [[ $OOM_RISK -gt 50 ]]; then
  WARN "High OOM risk detected (score: $OOM_RISK)"
  # Suggest standalone mode, memory limits, etc.
fi
```

Nuxt SSR/SPA Mismatch Detection

Module: brain.nuxt.render.sh

```bash
# Detect rendering mode inconsistencies
if [[ -f "nuxt.config.js" ]] || [[ -f "nuxt.config.ts" ]]; then
  # Check SSR configuration
  SSR_CONFIG=$(grep -E "ssr:\s*(true|false)" nuxt.config.*)

  # Check deployment target
  TARGET=$(grep "target:" nuxt.config.* | cut -d"'" -f2)

  if [[ "$TARGET" == "static" ]] && [[ "$SSR_CONFIG" =~ "true" ]]; then
    WARN "SSR enabled with static target - may cause issues"
  fi

  # Check for hydration mismatches
  HYDRATION_WARNINGS=$(grep -r "Hydration" .nuxt/dist/client/ 2>/dev/null | wc -l)
  if [[ $HYDRATION_WARNINGS -gt 0 ]]; then
    WARN "$HYDRATION_WARNINGS hydration warnings detected"
  fi
fi
```

Repository Hygiene

Dead File Detection

Module: `brain.hygiene.deadfiles.sh`

```bash
```

```bash
# Detect unused files
find_dead_files() {
  local IMPORT_PATTERNS=(
    "import.*from ['\"](.+)['\"]"
    "require\(['\"](.+)['\"]\)"
    "import ['\"](.+)['\"]"
  )

  # Build import graph
  for file in $(find src/ -name "*.js" -o -name "*.ts" -o -name "*.jsx" -o -name "*.tsx"); do
    for pattern in "${IMPORT_PATTERNS[@]}"; do
      grep -oE "$pattern" "$file" | sed "s/.*['\"]//g;s/['\"].*//g"
    done
  done | sort -u > /tmp/imported-files.txt

  # Find all source files
  find src/ -type f \( -name "*.js" -o -name "*.ts" -o -name "*.jsx" -o -name "*.tsx" \) \
    | sed 's|^src/||g;s|\.[jt]sx\?$||g' \
    | sort -u > /tmp/all-files.txt

  # Compare
  comm -23 /tmp/all-files.txt /tmp/imported-files.txt > dead-files.txt

  if [[ -s dead-files.txt ]]; then
    WARN "$(wc -l < dead-files.txt) potentially unused files detected"
    head -10 dead-files.txt
  fi
}
```

Orphaned Directories

Module: brain.hygiene.orphans.sh

```bash
```

```bash
# Detect empty or orphaned directories
find . -type d -empty ! -path "*/node_modules/*" ! -path "*/.git/*" | while read -r dir; do
  WARN "Empty directory: $dir"
done

# Detect test directories without tests
for test_dir in test/ tests/ __tests__; do
  if [[ -d "$test_dir" ]]; then
    TEST_COUNT=$(find "$test_dir" -name "*.test.*" -o -name "*.spec.*" | wc -l)
    if [[ $TEST_COUNT -eq 0 ]]; then
      WARN "Test directory exists but contains no tests: $test_dir"
    fi
  fi
done
```

UI/UX Constraints

Terminal Output Guidelines

Color Coding:

```bash
SUCCESS: Green ( ✅ )
WARNING: Yellow ( ⚠️ )
ERROR: Red ( ❌ )
INFO: Blue ( ℹ️ )
```

Progress Indicators:

```bash
```

```bash
# Spinners for long operations
spinner() {
  local pid=$1
  local delay=0.1
  local spinstr='⠋⠙⠹⠸⠼⠴⠦⠧⠇⠏'
  while ps -p $pid > /dev/null; do
    local temp=${spinstr#?}
    printf " [%c]  " "$spinstr"
    spinstr=$temp${spinstr%"$temp"}
    sleep $delay
    printf "\b\b\b\b\b\b"
  done
}

# Progress bars
progress_bar() {
  local current=$1
  local total=$2
  local width=50
  local percentage=$((current * 100 / total))
  local filled=$((width * current / total))

  printf "\r["
  printf "%${filled}s" | tr ' ' '▓'
  printf "%$((width - filled))s" | tr ' ' '▒'
  printf "] %3d%%" $percentage
}
```

Output Formatting:

```bash
# Section headers
section_header() {
  echo "━━━
```

JSON Output Standards

Consistent Schema:

```json
{
  "version": "3.0",
  "timestamp": "ISO 8601",
  "status": "success|warning|error",
  "data": {},
  "errors": [],
  "warnings": [],
  "metadata": {}
}
```

Error Format:

```json
{
  "code": "ERROR_CODE",
  "message": "Human-readable message",
  "file": "path/to/file",
  "line": 42,
  "severity": "critical|high|medium|low",
  "suggestion": "How to fix"
}
```

GitHub PR/Issue Templates

PR Template:

```markdown
```

```
## 🏥 Repo Doctor Auto-Repair

### Summary
Brief description of repairs

### Changes Made
- [ ] File 1
- [ ] File 2

### Before/After Metrics
| Metric | Before | After |
|--------|--------|-------|
| Score | 78 | 85 |
| Issues | 12 | 3 |

### Verification
- [x] Build passes
- [x] Tests pass
- [x] No regressions

### Review Notes
Additional context for reviewers
```

Issue Template:

```
markdown
```

```
## 🚨 Repo Doctor Alert

### Issue Type
- [ ] Security
- [ ] Build
- [ ] Configuration
- [ ] Dependency

### Details
Description of issue

### Impact
How this affects the repository

### Suggested Fix
Recommended remediation

### Diagnosis
Link to diagnosis.json
```

Temporal Architecture

Execution Timeline

```
Push/PR Event
    ↓
  ≈30s: Environment Setup
    ↓
  ≈2m: Detection Layer (Phases 1-8)
    ↓
  ≈3m: Normalization Layer (Phases 9-13)
    ↓
  ≈1m: Security Layer (Phases 14-16)
    ↓
  ≈2m: Autonomy Layer (Phases 17-19)
    ↓
Total: ≈8-10 minutes for full pipeline
```

Caching Strategy

GitHub Actions Cache:

```yaml
- name: Cache Dependencies
  uses: actions/cache@v4
  with:
    path: |
      node_modules
      ~/.cargo
      target/
      lib/
    key: ${{ runner.os }}-deps-${{ hashFiles('**/package-lock.json', '**/Cargo.lock') }}
    restore-keys: |
      ${{ runner.os }}-deps-
```

Build Artifact Persistence:

```yaml
- name: Cache Build
  uses: actions/cache@v4
  with:
    path: |
      .next/cache
      out/
      dist/
    key: ${{ runner.os }}-build-${{ github.sha }}
```

Scheduled Operations

Daily Health Checks:

```yaml
schedule:
  - cron: '0 2 * * *' # 2 AM UTC
```

Weekly Deep Scans:

```yaml
schedule:
  - cron: '0 0 * * 0' # Sunday midnight
```

Monthly Genome Updates:

```yaml
schedule:
  - cron: '0 0 1 * *' # First of month
```

Fleet Governance

Org-Wide Rollout

Bootstrap Script:

```bash
#!/bin/bash
# fleet-bootstrap.sh

ORG_DIR="/path/to/organization"
REPOS=$(find "$ORG_DIR" -maxdepth 1 -type d -name "*")

for repo in $REPOS; do
  if [[ -d "$repo/.git" ]]; then
    echo "Installing Repo Doctor in: $(basename $repo)"

    # Copy brain modules
    mkdir -p "$repo/repo-brain"
    cp -r repo-brain/* "$repo/repo-brain/"

    # Copy workflows
    mkdir -p "$repo/.github/workflows"
    cp .github/workflows/brain*.yml "$repo/.github/workflows/"

    # Initialize
    cd "$repo"
    bash repo-brain/brain.run.sh
    cd -
  fi
done

# Generate fleet report
bash repo-brain/brain.fleet.sh
```

Fleet Dashboard Schema

```
json
```

```
{
  "organization": "cyberai-network",
  "total_repositories": 47,
  "brain_version": "3.0",
  "last_sync": "2025-01-27T10:30:00Z",

  "health_distribution": {
    "GREEN": 32,
    "YELLOW": 12,
    "RED": 2,
    "AUTO": 1
  },

  "average_scores": {
    "overall": 84.3,
    "frontend": 87.2,
    "backend": 82.1,
    "web3": 79.8
  },

  "issue_summary": {
    "critical": 3,
    "high": 18,
    "medium": 67,
    "low": 142
  },

  "framework_distribution": {
    "next": 15,
    "angular": 8,
    "foundry": 12,
    "express": 7,
    "other": 5
  },

  "compliance": {
    "repos_with_ci": 45,
    "repos_with_tests": 41,
    "repos_with_greenlock": 32,
    "repos_with_copilot_instructions": 38
  },

  "top_issues": [
```

```
    {
      "type": "Outdated dependencies",
      "count": 23,
      "severity": "medium"
    },
    {
      "type": "Missing TypeScript",
      "count": 15,
      "severity": "low"
    },
    {
      "type": "Vercel OOM risk",
      "count": 8,
      "severity": "high"
    }
  ],

  "trends": {
    "score_change_7d": +3.2,
    "issues_resolved_7d": 47,
    "new_issues_7d": 12
  }
}
```

Customization Guide

Adding Custom Language Detection

Step 1: Add to `brain.detect.sh`

```bash
```

```bash
# Add Go detection
detect_go() {
  if [[ -f "go.mod" ]]; then
    echo "go" >> /tmp/languages.txt

    # Extract Go version
    GO_VERSION=$(grep "^go " go.mod | cut -d' ' -f2)

    # Check for common Go frameworks
    if grep -q "gin-gonic/gin" go.mod; then
      echo "gin" >> /tmp/frameworks.txt
    fi

    if grep -q "gorilla/mux" go.mod; then
      echo "gorilla" >> /tmp/frameworks.txt
    fi
  fi
}

# Add to main detection loop
detect_go
```

Step 2: Add CI contract to `brain.frameworks.ci.sh`

```bash
if [[ "$PRIMARY_LANG" == "go" ]]; then
  BUILD_COMMAND="go build ./..."
  TEST_COMMAND="go test ./... -v -race -coverprofile=coverage.out"
  OUTPUT_DIR="bin/"
fi
```

Step 3: Create canonical workflow template

```yaml
```

```yaml
# .github/workflows/go-ci.yml
name: Go CI

on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Go
        uses: actions/setup-go@v5
        with:
          go-version: '1.21'

      - name: Build
        run: go build ./...

      - name: Test
        run: go test ./... -v -race -coverprofile=coverage.out

      - name: Coverage
        run: go tool cover -html=coverage.out -o coverage.html
```

Adding Custom Security Patterns

Step 1: Add to `brain.ai.guard.sh`

```bash
```

```bash
# Custom API key pattern
CUSTOM_API_PATTERN='custom-api-[a-zA-Z0-9]{32}'

# Add to scan
grep -rn -E "$CUSTOM_API_PATTERN" . \
  --include="*.js" --include="*.ts" --include="*.env*" \
  --exclude-dir=node_modules \
  >> security-findings.txt

# Generate finding
if [[ -s security-findings.txt ]]; then
  echo '{
    "severity": "CRITICAL",
    "type": "secret_leak",
    "pattern": "Custom API Key",
    "files": "'$(cat security-findings.txt)'",
    "recommendation": "Move to environment variable"
  }' >> findings.json
fi
```

Creating Custom Repair Modules

Template:

```bash
```

```bash
#!/bin/bash
# brain.custom.repair.sh

set -euo pipefail

REPO_ROOT=$(pwd)

log() {
  echo "[brain.custom.repair] $1"
}

main() {
  log "Starting custom repair..."

  # 1. Detect issue
  if [[ ! -f "custom-config.json" ]]; then
    log "Missing custom-config.json"

    # 2. Apply fix
    cat > custom-config.json << 'EOF'
{
  "version": "1.0",
  "enabled": true
}
EOF

    log "Created custom-config.json"
  fi

  # 3. Verify fix
  if jq '.' custom-config.json >/dev/null 2>&1; then
    log "✅ Custom configuration valid"
  else
    log "❌ Custom configuration invalid"
    exit 1
  fi

  log "Custom repair complete"
}

main
```

Integration:

Add to `brain.run.sh`:

```bash
run_phase "Custom Repair" "brain.custom.repair.sh" false
```

Troubleshooting

Common Issues

1. Permission Denied Errors

Symptom:

```
bash: ./repo-brain/brain.run.sh: Permission denied
```

Fix:

```bash
chmod +x repo-brain/*.sh
```

Prevention: Phase 12 (brain.fix.safe.sh) automatically fixes this

---

2. Submodule Initialization Failures

Symptom:

```
fatal: No url found for submodule path 'lib/forge-std'
```

Fix:

```bash
git submodule update --init --recursive
```

Prevention: Phase 07 (brain.forge.repair.sh) handles this for Foundry projects

---

3. Build OOM Errors

Symptom:

```
FATAL ERROR: Reached heap limit Allocation failed - JavaScript heap out of memory
```

Fix:

```bash
# Increase Node.js memory
export NODE_OPTIONS="--max-old-space-size=4096"
npm run build

# Enable standalone mode for Next.js
# (Phase 10 does this automatically)
```

---

4. Merge Conflict Detection

Symptom:

```
❌ Merge conflict markers detected
```

Fix:

```bash
# Find conflicts
grep -r "<<<<<<< HEAD" .

# Resolve manually
git add <resolved-files>
git commit
```

---

5. Failed Security Scans

Symptom:

```
🚨 CRITICAL: API key detected in src/config.ts
```

Fix:

```bash
# 1. Remove from file
# 2. Add to .gitignore
echo ".env*" >> .gitignore

# 3. Remove from git history
git filter-branch --force --index-filter \
  'git rm --cached --ignore-unmatch src/config.ts' \
  --prune-empty --tag-name-filter cat -- --all

# 4. Use environment variables
echo "API_KEY=your-key-here" >> .env.local
```

Database-Specific Issues

Schema Migration Tracking

Custom Module: `brain.db.migrations.sh`

```bash
#!/bin/bash
# Detect uncommitted migrations

if [[ -d "prisma/migrations" ]]; then
  UNCOMMITTED=$(git ls-files --others --exclude-standard prisma/migrations/)

  if [[ -n "$UNCOMMITTED" ]]; then
    WARN "Uncommitted migrations detected"
    echo "$UNCOMMITTED"

    # Auto-commit if safe
    if [[ $(echo "$UNCOMMITTED" | wc -l) -eq 1 ]]; then
      git add prisma/migrations/
      git commit -m "db: Add migration $(basename $UNCOMMITTED)"
    fi
  fi
fi
```

Connection String Security

```bash
# Detect hardcoded database URLs
grep -rn "postgresql://.*:.*@" . \
  --include="*.js" --include="*.ts" --include="*.env.example" \
  --exclude-dir=node_modules

# Recommendation
cat << EOF
⚠️  Database connection string found in code

FIX:
1. Move to .env file
2. Add .env to .gitignore
3. Use environment variables:
   DATABASE_URL=${DATABASE_URL}
EOF
```

UI/UX Troubleshooting

React Hydration Mismatches

Detection:

```bash
# Check build logs for hydration warnings
grep -i "hydration" build.log

# Check for common causes
# 1. Date/time rendering
grep -rn "new Date()" --include="*.jsx" --include="*.tsx"

# 2. Random values
grep -rn "Math.random()" --include="*.jsx" --include="*.tsx"

# 3. Browser-only APIs in SSR
grep -rn "window\." --include="*.jsx" --include="*.tsx"
```

Remediation:

```typescript
```

```
// Use useEffect for browser-only code
import { useEffect, useState } from 'react';

function ClientOnly({ children }) {
  const [mounted, setMounted] = useState(false);

  useEffect(() => {
    setMounted(true);
  }, []);

  if (!mounted) return null;
  return <>{children}</>;
}
```

CSS Build Failures

Detection:

```bash
# Tailwind not compiling
if ! grep -q "@tailwind" src/styles.css; then
  WARN "Tailwind directives missing"
fi

# PostCSS config missing
if [[ ! -f "postcss.config.js" ]]; then
  WARN "PostCSS configuration missing"
fi
```

Auto-Fix:

```javascript
// postcss.config.js
module.exports = {
  plugins: {
    tailwindcss: {},
    autoprefixer: {},
  },
}
```

Implementation Checklist

Initial Setup

- [ ] Clone Repo Doctor repository
- [ ] Copy `repo-brain/` to your repository
- [ ] Copy `.github/workflows/brain*.yml` to your repository
- [ ] Run `chmod +x repo-brain/*.sh`
- [ ] Run `./repo-brain/brainctl.sh diagnose`
- [ ] Review `diagnosis.json`
- [ ] Address critical issues
- [ ] Run `./repo-brain/brainctl.sh run`
- [ ] Commit changes
- [ ] Push to trigger CI

Ongoing Maintenance

- [ ] Review diagnosis.json weekly
- [ ] Monitor GitHub Actions runs
- [ ] Respond to auto-created PRs within 48 hours
- [ ] Update brain modules monthly
- [ ] Run fleet sync for multi-repo setups
- [ ] Verify green lock status
- [ ] Update Copilot instructions after major changes

Fleet Deployment

- [ ] Install in one pilot repository
- [ ] Verify full pipeline
- [ ] Run `fleet-bootstrap.sh` for org-wide rollout
- [ ] Configure organization-level secrets
- [ ] Set up fleet monitoring dashboard
- [ ] Establish governance policies
- [ ] Train team on brainctl commands
- [ ] Document custom modifications

Compliance & Certification

Green Lock Certification

A repository achieves **Green Lock** status when:

1. ✅ All 19 phases complete successfully
2. ✅ `diagnosis.json` reports state: "GREEN"
3. ✅ Health score ≥ 90/100
4. ✅ Zero critical or high-severity issues
5. ✅ Build and tests pass
6. ✅ No security vulnerabilities
7. ✅ SHA-256 genome lock applied
8. ✅ Copilot instructions generated
9. ✅ Fleet telemetry updated

Verification Command:

```bash
./repo-brain/brainctl.sh verify
```

Repo Doctor Compliant Badge

Add to your README.md:

```markdown
[![Repo Doctor](https://img.shields.io/badge/Repo%20Doctor-GREEN-success)](./diagnosis.json)
```

API Reference

Environment Variables

```bash
```

```
# Required
GITHUB_TOKEN       # For PR creation

# Optional
NODE_OPTIONS       # Node.js memory settings
FOUNDRY_VERSION    # Foundry toolchain version
RUST_VERSION       # Rust toolchain version
```

Exit Codes

```bash
0  # Success
1  # General error
2  # Configuration error
3  # Security issue detected
4  # Build failure
5  # Test failure
10 # Critical phase failed
```

File Artifacts

```bash
diagnosis.json            # Health report
brain.fleet.json          # Fleet telemetry
.repo-brain.lock          # Integrity lock
build.log                 # Build output
test.log                  # Test output
security-findings.json    # Security scan results
.github/copilot-instructions.md  # AI governance
```

Appendices

Appendix A: Glossary

MERMEDA: Modular Extensible Repository Management, Enhancement, Detection, and Automation

Genome: The complete set of brain modules and configurations

Phase: A discrete step in the admission pipeline

Green Lock: Certification of repository health with SHA-256 seal

Forensic Security: Deep scanning for vulnerabilities

Telemetry: Fleet-wide metrics and statistics

Auto-Wiring: Autonomous PR creation and repair

Neural Contract: AI governance instructions for Copilot

Graduation: Successful completion of all phases

Appendix B: Version History

v3.0 (Current)

- 29-phase MERMEDA pipeline
- Enhanced Web3 support
- Fleet governance
- Auto-PR creation
- DB migration tracking

v2.0

- 19-phase operational loop
- GitHub Actions integration
- Foundry support

v1.0

- Initial release
- Basic detection and repair

Appendix C: Contributing

To contribute to Repo Doctor:

1. Fork the repository
2. Create feature branch

3. Add tests for new modules
4. Update documentation
5. Submit PR with clear description
6. Ensure all phases pass

Module Development Guidelines:

- Idempotent operations
- Clear error messages
- JSON output for data
- Security-first approach
- Comprehensive logging

Appendix D: License

Repo Doctor is built by **CyberAI Oracle Network**

Contact: V3@CyberAi.network

END OF REFERENCE DOCUMENT

Last Updated: 2025-01-27
Version: 3.0
Total Pages: 100+